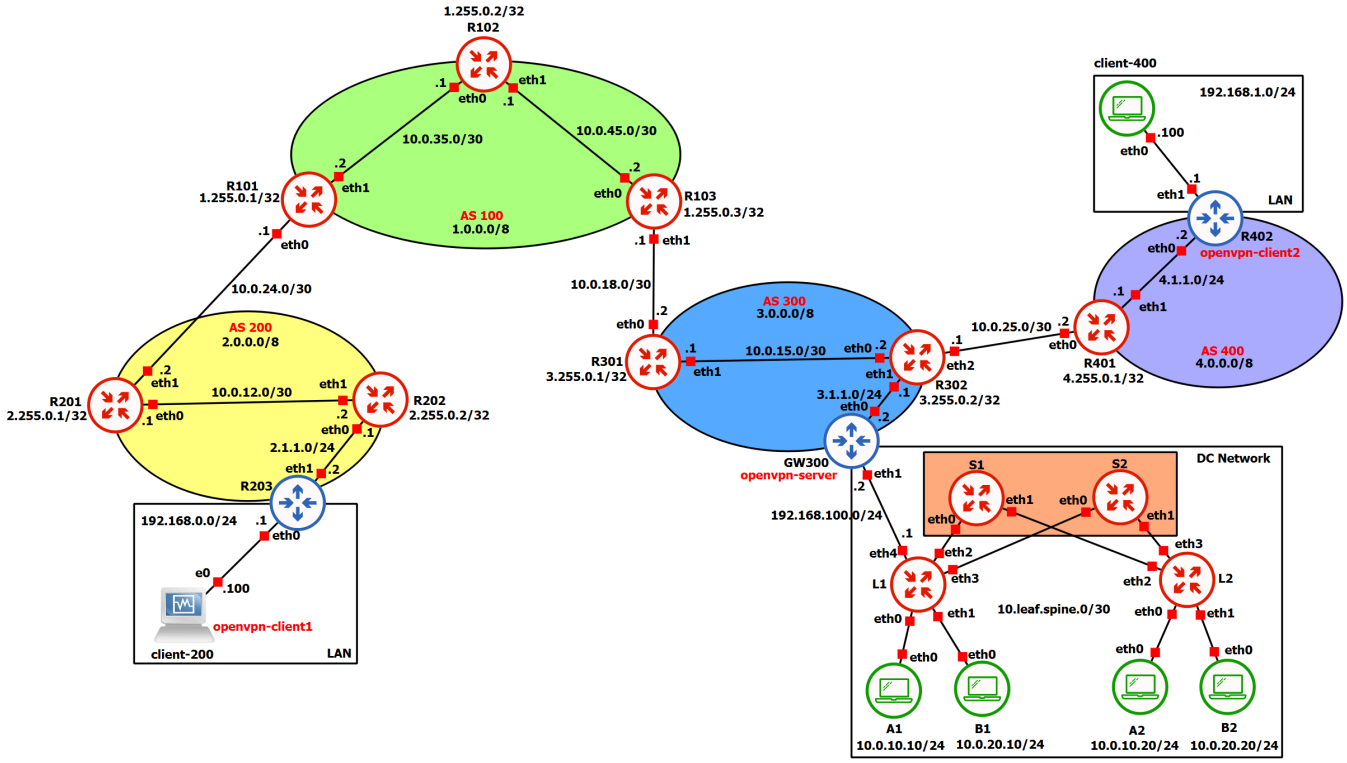
Network And System Defence - Project 1

Matteo Pallagrosi - 0338469
Università degli studi di Roma Tor Vergata
Ingegneria Informatica



Nell'immagine è mostrata la topologia di rete descritta nelle specifiche, con gli indirizzi utilizzati sui vari nodi e link presenti al suo interno.

Configurazione su GNS3

Sulle diverse tipologie di nodi presenti all'interno del progetto sono state configurate directory persistenti e script lanciati automaticamente all'avvio così da automatizzare la configurazione dell'intera topologia al lancio.

- Sui nodi FRR (nodi rossi) la directory `root` e la directory `/etc/frr` sono state impostate come persistenti. All'interno della root è stato creato il file start.sh, eseguito all'avvio del nodo, che si occupa (oltre ad avviare i demoni per le configurazioni di BGP, OSPF, LDP) di lanciare sul terminale di FRR lo script `/etc/frr/net.sh` tramite il comando `vtysh < /etc/frr/net.sh`. Sui nodi R101, R102 e R103 questo script ha anche il compito di impostare la configurazione necessaria al corretto funzionamento del labeling sulle interfacce di questi nodi afferenti ad una rete MPLS, tramite i comandi `cp /etc/frr/sysctl.conf /etc/sysctl.conf` e `sysctl -p`. Sui nodi FRR della DC Network

lo script `start.sh` si occupa anche di lanciare `net.sh` (presente sempre nella root directory), per la configurazione di rete (interfacce di rete, bridge, vrf) da realizzare sulla bash linux.

- Sui Linux routers (nodi blu) la directory `root` è persistente. E' presente lo script `start.sh` eseguito all'avvio del nodo, con il compito di eseguire `net.sh`, per la configurazione delle interfaccie di rete, route, e dove necessario firewall con iptables.
- Sui nodi client (nodi verdi) l'approccio adottato è analogo a quello dei Linux routers per la configurazione di rete.

Vengono quindi mostrate le configurazioni dei nodi presenti all'interno della topologia.

BGP

In questa sezione vengono mostrate le configurazioni per il setup delle interfacce di rete, del peering BGP (internal e external), del routing interno agli AS gestito con OSPF, e per l'AS100 della rete MPLS e del LDP.

AS100

MPLS pre-setup
Su **R101**:
in /etc/frr/sysctl.conf

```
net.mpls.conf.lo.input = 1
net.mpls.conf.eth1.input = 1
net.mpls.platform_labels = 100000
```

#cp /etc/frr/sysctl.conf /etc/sysctl.conf
save and issue “sysctl -p”

Su **R102**:
in /etc/frr/sysctl.conf

```
net.mpls.conf.lo.input = 1
net.mpls.conf.eth0.input = 1
net.mpls.conf.eth1.input = 1
net.mpls.platform_labels = 100000
```

#cp /etc/frr/sysctl.conf /etc/sysctl.conf
save and issue “sysctl -p”

Su **R103**:
in /etc/frr/sysctl.conf

```
net.mpls.conf.lo.input = 1
net.mpls.conf.eth0.input = 1
net.mpls.platform_labels = 100000
```

#cp /etc/frr/sysctl.conf /etc/sysctl.conf
save and issue “sysctl -p”

IP address, BGP peering, OSPF, MPLS LDP
Questo nodo presenta relazione di peering eBGP con R201 (AS 200).
Su **R101**:

```
conf t
interface lo
ip address 1.255.0.1/32

interface eth0
ip address 10.0.24.1/30

interface eth1
ip address 10.0.35.2/30

router ospf
router-id 1.255.0.1
network 1.255.0.1/32 area 0
network 10.0.35.0/30 area 0
exit

mpls ldp
router-id 1.255.0.1
ordered-control
address-family ipv4
discovery transport-address 1.255.0.1
interface eth1
interface lo
exit
```

```
router bgp 100
neighbor 1.255.0.3 remote-as 100
neighbor 1.255.0.3 update-source 1.255.0.1
neighbor 10.0.24.2 remote-as 200

address-family ipv4
neighbor 1.255.0.3 activate
neighbor 1.255.0.3 next-hop-self
redistribute static
exit

end
```

Su **R102**:

```
conf t
interface lo
ip address 1.255.0.2/32

interface eth0
ip address 10.0.35.1/30

interface eth1
ip address 10.0.45.1/30

router ospf
router-id 1.255.0.2
network 1.255.0.2/32 area 0
network 10.0.35.0/30 area 0
network 10.0.45.0/30 area 0
exit

mpls ldp
router-id 1.255.0.2
ordered-control
address-family ipv4
discovery transport-address 1.255.0.2
```

```
interface eth0
interface eth1
interface lo
exit

end
```

Si evidenzia come questo nodo non presenti alcuna configurazione per BGP, in quanto svolge solo il ruolo di Label Switching Router, con funzionalità di swap delle label e forwarding dei pacchetti all'interno della rete MPLS.

Questo nodo presenta una relazione di peering eBGP con R301 (AS 300).
Su **R103**:

```
conf t
interface lo
ip address 1.255.0.3/32

interface eth0
ip address 10.0.45.2/30

interface eth1
ip address 10.0.18.1/30

router ospf
router-id 1.255.0.3
network 1.255.0.3/32 area 0
network 10.0.45.0/30 area 0
exit

mpls ldp
router-id 1.255.0.3
ordered-control
address-family ipv4
discovery transport-address 1.255.0.3
interface eth0
interface lo
exit
```

```
router bgp 100
neighbor 1.255.0.1 remote-as 100
neighbor 1.255.0.1 update-source 1.255.0.3
neighbor 10.0.18.2 remote-as 300

address-family ipv4
neighbor 1.255.0.1 activate
neighbor 1.255.0.1 next-hop-self
redistribute static
exit

end
```

AS 200

IP address, BGP peering, OSPF
Questo nodo presenta relazione di peering eBGP con R101 (AS 100).
Su **R201**:

```
conf t
interface lo
ip address 2.255.0.1/32

interface eth0
ip address 10.0.12.1/30

interface eth1
ip address 10.0.24.2/30

router ospf
router-id 2.255.0.1
network 2.255.0.1/32 area 0
network 10.0.12.0/30 area 0
exit

router bgp 200
neighbor 2.255.0.2 remote-as 200
neighbor 2.255.0.2 update-source 2.255.0.1
neighbor 2.255.0.2 next-hop-self
```

```
neighbor 10.0.24.1 remote-as 100
exit

end
```

Su **R202**:

```
conf t
interface lo
ip address 2.255.0.2/32

interface eth0
ip address 2.1.1.1/24

interface eth1
ip address 10.0.12.2/30

router ospf
router-id 2.255.0.2
network 2.255.0.2/32 area 0
network 2.1.1.0/24 area 0
network 10.0.12.0/30 area 0
exit

router bgp 200
neighbor 2.255.0.1 remote-as 200
neighbor 2.255.0.1 update-source 2.255.0.2
neighbor 2.255.0.1 next-hop-self
network 2.1.1.0/24
exit

end
```

IP address, NAT, firewall
Questo nodo non rappresenta un router BGP, ma presenta una route di default verso R202, e rappresenta l'access gateway per la LAN 192.168.0.0/24, realizzando NAT dinamico. Presenta

inoltre un firewall per permettere il passaggio di pacchetti afferenti esclusivamente a connessioni stabilite a partire dalla LAN.
Su **R203**:

```
# enable ip forwarding
sysctl -w net.ipv4.ip_forward=1

ip addr add 192.168.0.1/24 dev eth0
ip addr add 2.1.1.2/24 dev eth1
ip route add default via 2.1.1.1

# NAT
iptables -t nat -A POSTROUTING -o eth1 -j MASQUERADE

#Firewall
export LAN=eth0
export WAN=eth1

iptables -F
iptables -P FORWARD DROP
iptables -P INPUT DROP
iptables -P OUTPUT ACCEPT

# Propaga solo pacchetti su connessioni stabilite dalla LAN
iptables -A FORWARD -m state --state ESTABLISHED -j ACCEPT
iptables -A FORWARD -i $LAN -o $WAN -j ACCEPT

# Dalla LAN posso mandare pacchetti in input all'AP
iptables -A INPUT -m state --state ESTABLISHED -j ACCEPT
iptables -A INPUT -i $LAN -j ACCEPT
```

Questo nodo è una macchina virtuale Lubuntu 24.04. La configurazione di rete è stata quindi realizzata tramite le impostazioni di rete dell'interfaccia a disposizione, equivalente ai seguenti comandi.
Su **client-200**:

```
ip addr add 192.168.0.100/24 dev eth0
```

```
ip route add default via 192.168.0.1
```

Configurazione Falco su client-200
Su questa macchina è stato configurato Falco per il monitoraggio di attività potenzialmente malevoli. In particolare si monitora l'accesso a file e directory ritenute "sensibili", quali:

- /etc/shadow -> contenente le password hashate degli utenti del sistema.
- /etc/passwd -> con informazioni degli utenti del sistema.
- /etc/hosts -> per le risoluzioni note di nomi host in indirizzi IP. Un accesso non desiderato potrebbe avere come obiettivo quello di iniettare un mapping malevolo al suo interno.
- /etc/gshadpow -> password hashate dei gruppi.
- /etc/sudoers -> specifica chi può eseguire comandi come sudo.
- /boot/grub/grub.cfg -> permette di specificare comandi da eseguire all'avvio. Una sua manomissione può quindi compromettere l'avvio del sistema.
- directory /etc/ssh -> contenente info di configurazione ssh e chiavi pubbliche e private.
- directory /etc/systemd -> per la configurazione dei servizi di sistemi.
  Viene inoltre monitorato ogni tentativo di connessione in uscita a indirizzi IP ritenuti malevoli. La lista, aggiornata giornalmente, degli indirizzi IP utilizzata è presente in https://raw.githubusercontent.com/stamparm/ipsum/master/ipsum.txt. Tramite il seguente comando la lista è stata scaricata e formattata come un file YAML:

```
sudo sh -c "curl --compressed
https://raw.githubusercontent.com/stamparm/ipsum/master/ipsum.txt 2>/dev/null
| grep -v '#' | grep -v -E 's[1-5]$' | cut -f 1 | sed 's/.*/\"&\",/g' | tr
'\n' ' ' | sed 's/, $//' | sed 's/.*/- list: unsafe_ips\n  items: [&]/' >
/etc/falco/unsafe_ips_list.yaml"
```

Data l'assenza di una vera e propria connessione a Internet (non è presente un nodo "NAT" nella topologia di rete) non è possibile rieseguire il comando periodicamente (ad esempio tramite uno script) per aggiornare in modo automatico la lista degli IP malevoli. Ciò nonostante per dimostrare l'efficacia del sistema di monitoraggio si è scelto di utilizzare una lista di IP unsafe "reale".

Nel file di configurazione `/etc/falco/falco.yaml` sono state disabilitate le regole di default (contenute in `/etc/falco/falco_rules.yaml`). Nella directory `/etc/falco` è presente il file `falco_rules.local.yaml` con il seguente contenutom che abilita le regole sopra descritte:

```
- macro: sensitive_files
  condition: fd.name in ("/etc/shadow", "/etc/passwd",
"/etc/hosts","/etc/gshadow", "/etc/sudoers", "/boot/grub/grub.cfg")
```

```
- macro: open_syscalls
  condition: evt.type in (open, openat, openat2)

- rule: Unauthorized Access to Sensitive Files
  desc: Detect unauthorized access attempts to sensitive system files
  condition: open_syscalls and sensitive_files
  exceptions:
  - name: sudo_su
    fields: proc.name
    values: ["sudo", "su"]
  output: "Unauthorized access to %fd.name by user=%user.name
(command=%proc.cmdline)"
  priority: WARNING
  tags: [filesystem, authentication]

- macro: sensitive_directories
  condition: fd.name pmatch (/etc/ssh, /etc/systemd)

- rule: Unauthorized Access to Sensitive Directories
  desc: Detect unauthorized access attempts to sensitive system directories
  condition: open_syscalls and sensitive_directories
  exceptions:
  - name: sudo_su
    fields: proc.name
    values: ["sudo", "su"]
  output: "Unauthorized access to %fd.name by user=%user.name
(command=%proc.cmdline)"
  priority: WARNING
  tags: [filesystem, authentication]

- macro: outbound
  condition: >
    (((evt.type = connect and evt.dir=<)) or
     (fd.typechar = 4 or fd.typechar = 6) and
     (fd.ip != "0.0.0.0" and fd.net != "127.0.0.0/8") and (evt.rawres >= 0 or
evt.res = EINPROGRESS))

- macro: malicious_ips
  condition: (fd.sip in (unsafe_ips) or fd.cip in (unsafe_ips))
```

```
- rule: Outbound connection to Unsafe IP
  desc: Detect outbound connections to IP addresses listed as unsafe
  condition: outbound and malicious_ips
  output: "Suspicious connection to unsafe IP detected (command=%proc.cmdline
connection=%fd.name user=%user.name)"
  priority: WARNING
  tags: [network, malicious]
```

Per verificare lo stato di Falco è possibile utilizzare il seguente comando:
`sudo systemctl status falco-modern-bpf.service`

Per controllare i messaggi di warning prodotti durante il monitoraggio è possibile consultare l'apposito log file con:
`sudo cat /var/log/syslog`

AS 300

IP address, BGP peering, OSPF
Questo nodo presenta una relazione di peering eBGP con R103 (AS 100).
Su **R301**:

```
conf t
interface lo
ip address 3.255.0.1/32

interface eth0
ip address 10.0.18.2/30

interface eth1
ip address 10.0.15.1/30

router ospf
router-id 3.255.0.1
network 3.255.0.1/32 area 0
network 10.0.15.0/30 area 0
exit

router bgp 300
neighbor 3.255.0.2 remote-as 300
neighbor 3.255.0.2 update-source 3.255.0.1
```

```
neighbor 3.255.0.2 next-hop-self
neighbor 10.0.18.1 remote-as 100
exit

end
```

Questo nodo presenta una relazione peering eBGP con R401 (AS 400).
Su **R302**:

```
conf t
interface lo
ip address 3.255.0.2/32

interface eth0
ip address 10.0.15.2/30

interface eth1
ip address 3.1.1.1/24

interface eth2
ip address 10.0.25.1/30

router ospf
router-id 3.255.0.2
network 3.255.0.2/32 area 0
network 3.1.1.0/24 area 0
network 10.0.15.0/30 area 0
exit

router bgp 300
neighbor 3.255.0.1 remote-as 300
neighbor 3.255.0.1 update-source 3.255.0.2
neighbor 3.255.0.1 next-hop-self
neighbor 10.0.25.2 remote-as 400
network 3.1.1.0/24
exit

end
```

IP address, NAT

Questo nodo non rappresenta un BGP speaker, ma mantiene una route di default verso R302, e agisce da access gateway per la Data Center Network, realizzando quindi il NAT dinamico. Su di esso sono configurate due interfacce virtuali su eth1 per il routing dei pacchetti afferenti ai tenant A e B del Data Center. Inoltre i pacchetti diretti al tenant B (raggiungibile tramite OpenVPN come mostrato più avanti) vengono mascherati per permettere a B1 e B2 di rispondere alle richieste ricevute (poiché questi client conoscono solo come raggiungere GW300 e non le reti esterne).

Su **GW300**:

```
# enable ip forwarding
sysctl -w net.ipv4.ip_forward=1

ip addr add 3.1.1.2/24 dev eth0
ip route add default via 3.1.1.1

#creazione bridge per gestire pacchetti del tenantA
ip link add vrf1 type vrf table 1100
ip link set vrf1 up
ip link add br10 type bridge
ip link set br10 master vrf1
ip link set br10 addr aa:bb:cc:00:00:02
ip addr add 192.168.100.2/24 dev br10
ip link add link eth1 name eth1.10 type vlan id 10
ip link set eth1.10 master br10
ip link set eth1.10 up
ip link set br10 up
ip route add 10.0.10.0/24 via 192.168.100.1 dev br10

#creazione bridge per gestire pacchetti del tenantB
ip link add vrf2 type vrf table 1200
ip link set vrf2 up
ip link add br20 type bridge
ip link set br20 master vrf2
ip link set br20 addr aa:bb:cc:00:00:03
ip addr add 192.168.100.2/24 dev br20
ip link add link eth1 name eth1.20 type vlan id 20
ip link set eth1.20 master br20
ip link set eth1.20 up
```

```
ip link set br20 up
ip route add 10.0.20.0/24 via 192.168.100.1 dev br20

# NAT
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

iptables -t nat -A POSTROUTING -o br20 ! -s 10.0.10.0/24 -j MASQUERADE
```

AS 400

IP address, BGP peering, OSPF
Questo nodo presenta una relazione di peering eBGP con R302 (AS 300).
Su **R401**:

```
conf t
interface lo
ip address 4.255.0.1/32

interface eth0
ip address 10.0.25.2/30

interface eth1
ip address 4.1.1.1/24

router bgp 400
neighbor 10.0.25.1 remote-as 300
network 4.1.1.0/24
exit

end
```

IP address, NAT
Questo nodo non rappresenta un BGP speaker, ma presenta una route di default verso R401, e agisce come access gateway per la LAN 192.168.1.0/24, realizzando il NAT dinamico.
Su **R402**:

```
# enable ip forwarding
sysctl -w net.ipv4.ip_forward=1
```

```
ip addr add 4.1.1.2/24 dev eth0
ip addr add 192.168.1.1/24 dev eth1
ip route add default via 4.1.1.1

# NAT
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

Su questo nodo si configura l'indirizzo di rete e la route di default verso l'access gateway.
Su **client-400**:

```
ip addr add 192.168.1.100/24 dev eth0
ip route add default via 192.168.1.1
```

DC Network

Per la realizzazione di questa rete si utilizza una topologia leaf-spine.
Si evidenzia che i client dei diversi tenants sono collegati a interfacce fisiche distinte sulle leaf, e per tale motivo è possibile associare direttamente ogni interfaccia fisica alla vrf da usare per quel tenant, senza dover creare interfacce virtuali a cui associare delle vlan. Le interfacce virtuali sono create su eth4 verso l'access gateway, per permettere di distinguere i pacchetti provenienti dal tenant A o dal tenant B.

VRF, L2VNI, L3VNI
Su questo nodo vengono configurate le interfacce di rete, vrf e bridges da utilizzare per il tenant A e il tenant B, oltre agli identificativi L2VNI per distinguere i domini di broadcast dei diversi tenant. Vengono inoltre configurati gli L3VNI, per permettere a ciascun tenant di propapagare pacchetti tra i diversi domini di broadcast da lui gestiti. In particolare sul tenant A viene configurata una route di default verso l'access gateway, sfruttando Type 5 route per la rete di default, permettendo quindi al tenant A di raggiungere tramite GW300 le reti pubbliche esterne alla DC network, invece il tenant B può propagare pacchetti verso l'access gateway, ma non verso l'esterno (questo gli permette di rispondere ai pacchetti ricevuti dai client della OpenVPN, ma non di propagare pacchetti verso le reti pubbliche esterne alla DC Network).
Su **L1**:

```
# vtep IP address
ip addr add 100.64.0.1/32 dev lo
```

```
# vrfs and bridges (tenantA)
ip link add vrf1 type vrf table 1100
ip link set vrf1 up
ip link add br10 type bridge
ip link set br10 master vrf1
ip link set br10 addr aa:bb:cc:00:00:01
ip addr add 10.0.10.1/24 dev br10
ip link set eth0 master br10
ip link set br10 up

# vni device (tenantA)
ip link add vni110 type vxlan local 100.64.0.1 dstport 4789 id 110 nolearning
ip link set vni110 master br10 addrgenmode none
ip link set vni110 type bridge_slave neigh_suppress on learning off
ip link set vni110 up

# vrfs and bridges (tenantB)
ip link add vrf2 type vrf table 1200
ip link set vrf2 up
ip link add br20 type bridge
ip link set br20 master vrf2
ip link set br20 addr aa:bb:cc:00:00:11
ip addr add 10.0.20.1/24 dev br20
ip link set eth1 master br20
ip link set br20 up

# vni device (tenantB)
ip link add vni120 type vxlan local 100.64.0.1 dstport 4789 id 120 nolearning
ip link set vni120 master br20 addrgenmode none
ip link set vni120 type bridge_slave neigh_suppress on learning off
ip link set vni120 up
ip link set br20 up

#L3VNI (tenantA)
ip link add br100 type bridge
ip link set br100 master vrf1 addrgenmode none
ip link set br100 addr aa:bb:cc:00:00:64
ip link add vni100 type vxlan local 100.64.0.1 dstport 4789 id 100 nolearning
ip link set vni100 master br100 addrgenmode none
ip link set vni100 type bridge_slave neigh_suppress on learning off
```

```
ip link set vni100 up
ip link set br100 up

ip link add link eth4 name eth4.10 type vlan id 10
ip link set eth4.10 master br100
ip link set eth4.10 up
ip link set eth4.10 master vrf1
ip addr add 192.168.100.1/24 dev eth4.10

#L3VNI (tenantB)
ip link add br200 type bridge
ip link set br200 master vrf2 addrgenmode none
ip link set br200 addr aa:bb:cc:00:00:70
ip link add vni200 type vxlan local 100.64.0.1 dstport 4789 id 200 nolearning
ip link set vni200 master br200 addrgenmode none
ip link set vni200 type bridge_slave neigh_suppress on learning off
ip link set vni200 up
ip link set br200 up

ip link add link eth4 name eth4.20 type vlan id 20
ip link set eth4.20 master br200
ip link set eth4.20 up
ip link set eth4.20 master vrf2
ip addr add 192.168.100.1/24 dev eth4.20
```

Questo nodo è configurato in maniera analoga al precedente.
Su **L2**:

```
# vtep IP address
ip addr add 100.64.0.2/32 dev lo

# vrfs and bridges (tenantA)
ip link add vrf1 type vrf table 1100
ip link set vrf1 up
ip link add br10 type bridge
ip link set br10 master vrf1
ip link set br10 addr aa:bb:cc:00:00:02
ip addr add 10.0.10.1/24 dev br10
```

```
ip link set eth0 master br10
ip link set br10 up

# vni device (tenantA)
ip link add vni110 type vxlan local 100.64.0.2 dstport 4789 id 110 nolearning
ip link set vni110 master br10 addrgenmode none
ip link set vni110 type bridge_slave neigh_suppress on learning off
ip link set vni110 up

# vrfs and bridges (tenantB)
ip link add vrf2 type vrf table 1200
ip link set vrf2 up
ip link add br20 type bridge
ip link set br20 master vrf2
ip link set br20 addr aa:bb:cc:00:00:22
ip addr add 10.0.20.1/24 dev br20
ip link set eth1 master br20
ip link set br20 up

# vni device (tenantB)
ip link add vni120 type vxlan local 100.64.0.2 dstport 4789 id 120 nolearning
ip link set vni120 master br20 addrgenmode none
ip link set vni120 type bridge_slave neigh_suppress on learning off
ip link set vni120 up

#L3VNI (tenant A)
ip link add br100 type bridge
ip link set br100 master vrf1 addrgenmode none
ip link set br100 addr aa:bb:cc:00:00:65
ip link add vni100 type vxlan local 100.64.0.2 dstport 4789 id 100 nolearning
ip link set vni100 master br100 addrgenmode none
ip link set vni100 type bridge_slave neigh_suppress on learning off
ip link set vni100 up
ip link set br100 up

# L3VNI(tenantB)
ip link add br200 type bridge
ip link set br200 master vrf2 addrgenmode none
ip link set br200 addr aa:bb:cc:00:00:71
ip link add vni200 type vxlan local 100.64.0.2 dstport 4789 id 200 nolearning
```

```
ip link set vni200 master br200 addrgenmode none
ip link set vni200 type bridge_slave neigh_suppress on learning off
ip link set vni200 up
ip link set br200 up
```

IP address, BGP peering, OSPF, EVPN
Sui nodi spine sono configurate le interfacce di rete, le relazioni di peering BGP e si utilizza EVPN come control plane per la discovery automatica delle VTEP e l'establishment automatico dei tunnel VXLAN.
Su **S1**:

```
conf t
interface eth0
ip address 10.1.1.2/30

interface eth1
ip address 10.2.1.2/30

interface lo
ip address 100.64.0.10/32

router ospf
ospf router-id 100.64.0.10
network 0.0.0.0/0 area 0
exit

router bgp 65000
bgp router-id 100.64.0.10
neighbor 10.1.1.1 remote-as 65001
neighbor 10.2.1.1 remote-as 65002

address-family l2vpn evpn
neighbor 10.1.1.1 activate
neighbor 10.2.1.1 activate
exit-address-family
exit
```

Su **S2**:

```
conf t
interface eth0
ip address 10.1.2.2/30

interface eth1
ip address 10.2.2.2/30

interface lo
ip address 100.64.0.11/32

router ospf
ospf router-id 100.64.0.11
network 0.0.0.0/0 area 0
exit

router bgp 65000
bgp router-id 100.64.0.11
neighbor 10.1.2.1 remote-as 65001
neighbor 10.2.2.1 remote-as 65002

address-family l2vpn evpn
neighbor 10.1.2.1 activate
neighbor 10.2.2.1 activate
exit-address-family
exit
```

Anche sui nodi leaf è configurato EVPN come control plane.
Il nodo leaf L1 annuncia le reti che permette di raggiungere, ossia la route di default (attraverso l'access gateway GW300) per il tenant A (vrf1) e l'access gateway per il tenant B (vrf2).
Su **L1**:

```
conf t
interface eth2
ip address 10.1.1.1/30

interface eth3
```

```
ip address 10.1.2.1/30

router ospf
ospf router-id 100.64.0.1
network 10.1.1.0/30 area 0
network 10.1.2.0/30 area 0
network 100.64.0.1/32 area 0
exit

router bgp 65001
neighbor 10.1.1.2 remote-as 65000
neighbor 10.1.2.2 remote-as 65000

address-family ipv4 unicast
network 100.64.0.1/32
exit

address-family l2vpn evpn
neighbor 10.1.1.2 activate
neighbor 10.1.2.2 activate
advertise-all-vni
advertise-svi-ip
exit-address-family
exit

router bgp 65001 vrf vrf1
address-family ipv4 unicast
redistribute static
exit-address-family

address-family l2vpn evpn
advertise ipv4 unicast
exit-address-family
exit

router bgp 65001 vrf vrf2
address-family ipv4 unicast
redistribute static
exit-address-family
```

```
address-family l2vpn evpn
advertise ipv4 unicast
exit-address-family
exit

vrf vrf1
ip route 0.0.0.0/0 192.168.100.2
vni 100
exit

router bgp 65001 vrf vrf1
address-family ipv4 unicast
network 0.0.0.0/0
exit-address-family
exit

vrf vrf2
vni 200
exit

router bgp 65001 vrf vrf2
address-family ipv4 unicast
network 192.168.100.0/24
exit-address-family
exit
```

La configurazione su L2 è analoga a L1, con la differenza che non deve originare alcuna rete. Su **L2**:

```
conf t
interface eth2
ip address 10.2.1.1/30

interface eth3
ip address 10.2.2.1/30

router ospf
ospf router-id 100.64.0.2
```

```
network 10.2.1.0/30 area 0
network 10.2.2.0/30 area 0
network 100.64.0.2/32 area 0
exit

router bgp 65002
neighbor 10.2.1.2 remote-as 65000
neighbor 10.2.2.2 remote-as 65000

address-family ipv4 unicast
network 100.64.0.2/32
exit

address-family l2vpn evpn
neighbor 10.2.1.2 activate
neighbor 10.2.2.2 activate
advertise-all-vni
advertise-svi-ip
exit-address-family
exit

router bgp 65002 vrf vrf1
address-family ipv4 unicast
redistribute static
exit-address-family

address-family l2vpn evpn
advertise ipv4 unicast
exit-address-family
exit

router bgp 65002 vrf vrf2
address-family ipv4 unicast
redistribute static
exit-address-family

address-family l2vpn evpn
advertise ipv4 unicast
exit-address-family
exit
```

```
vrf vrf1
vni 100
exit

vrf vrf2
vni 200
exit
```

---

clients
Infine sui client si configurano le interfacce di rete e le route di default verso i nodi leaf.
Su **A1**:

```
ip addr add 10.0.10.10/24 dev eth0
ip route add default via 10.0.10.1
```

Su **A2**:

```
ip addr add 10.0.10.20/24 dev eth0
ip route add default via 10.0.10.1
```

Su **B1**:

```
ip addr add 10.0.20.10/24 dev eth0
ip route add default via 10.0.20.1
```

Su **B2**:

```
ip addr add 10.0.20.20/24 dev eth0
ip route add default via 10.0.20.1
```

E' possibile verificare la corretta configurazione dei nodi della DC Network con i seguenti comandi:

`show ip route vrf vrf1` -> per visualizzare le informazioni di routing sulla vrf di un certo tenant.

`show bgp l2vpn evpn` -> per verificare le informazioni apprese tramite lo scambio dei messaggi realizzato dal control plane EVPN.

`show evpn vni` -> per visualizzare le vni configurate sul nodo.

OpenVPN

I nodi client-200 e R402 prendono parte come clients ad una vpn realizzata con OpenVPN, mentre GW300 svolge il ruolo di server. La vpn è configurata in modo da permettere una topologia di comunicazione hub and spoke, con i client che possono comunicare tra loro attraverso il server. La vpn permette inoltre di contattare la rete del tenant B (10.0.20.0/24) situata dietro a GW300 (non permette invece di contattare le macchine del tenant A).

Per la configurazione della vpn con OpenVPN sono stati seguiti i seguenti passi.
Su GW300 (openvpn-server) è stato generato, utilizzando il tool easy-rsa, il materiale crittografico con i seguenti comandi, eseguiti nelle directory indicate:

in /usr/share/easy-rsa

```
./easyrsa init-pki
./easyrsa build-ca nopass
```

Genera un certificato e una chiave privata per il server

```
./easyrsa build-server-full server nopass
```

Generate chiavi e certificati dei clients

```
./easyrsa build-client-full client1 nopass
./easyrsa build-client-full client2 nopass
```

I parametri Diffie Hellman devono essere generati per il server OpenVPN

```
./easyrsa gen-dh
```

A questo punto nella `root` directory (persistente) sono state create tre cartelle per separare il materiale crittografico:

```
mkdir server
mkdir client1
mkdir client2
```

Il materiale crittografico generato nella directory `/usr/share/easy-rsa` è stato quindi copiato nelle rispettive directory in `root`, secondo il seguente schema (i file .key contengono le chiavi private, i .crt sono i certificati pubblici):

- ca.crt (chiave pubblica della CA master) -> presente su tutti i nodi della vpn (server + tutti i clients) per verificare la firma di tutti i certificati (per verificare che siano emessi dalla CA master).
- ca.key (chiave privata della CA master) -> non è necessario mantenerla sul server, potrebbe essere memorizzata anche su un dispositivio di memorizzazione esterno.
- client1.key -> solo client1.
- client2.key -> solo client2.
- server.key -> solo server.
- client1.crt -> solo client1 (che poi manda durante l'handshake per autenticarsi al server. Il server valida questo certificato usando la chiave pubblica della CA master, di cui infatti possiede il certificato).
- client2.crt -> solo client2 (scambiato durante l'handshake).
- server.crt -> solo server (scambiato durante l'handshake).
- dh.pem (parametri Diffie Hellman)-> è mantenuto solo sul server

Ciascuna cartella contenente il materiale crittografico è stata quindi trasferita sul rispettivo nodo usando ssh:
client1 -- client-200
client2 -> R402
server -> GW300 (già presente).

VPN configuration
Infine sono stati definiti i parametri di configurazione, che permettono di installare sui vari nodi le route grazie al quale è possibile propagare pacchetti nella vpn. L'indirizzo di rete scelto per la vpn è 192.168.150.0/24.

Sul **GW300** (openvpn-server) dentro `root` è stato creato il file `server.ovpn` con il seguente contenuto:

```
port 1194
proto udp
dev tun
```

```
ca ca.crt
cert server.crt
key server.key
dh dh.pem
server 192.168.150.0 255.255.255.0
push "route 10.0.20.0 255.255.255.0"
push "route 192.168.1.0 255.255.255.0"
route 192.168.1.0 255.255.255.0
client-config-dir
ccd client-to-client
keepalive 10 120
cipher AES-256-GCM
```

Con i comandi push route viene influenzato l'underlay routing, permettendo di comunicare ai client che le reti 192.168.1.0/24 (LAN dietro a R402) e 10.0.20.0 (tenant B) sono raggiungibili tramite la vpn.

Sempre su **GW300** è stata creata una directory `ccd` contenente il file `client2`:

```
iroute 192.168.1.0 255.255.255.0
```

Questo comando influenza l'overlay routing, dichiarando che la rete 192.168.1.0/24 è raggiungibile tramite il tunnel stabilito dal server con openvpn-client2 (R402).

Su **client-200** (openvpn-client1) è stato creato il file `client1.ovpn`:

```
client
dev tun
proto udp
remote 3.1.1.2 1194
resolv-retry infinite
ca ca.crt
cert client1.crt
key client1.key
remote-cert-tls server
cipher AES-256-GCM
```

Con questo file di configurazione si definisce il server (indirizzo e porta) della vpn, gli algoritmi e il materiale crittografico per l'autenticazione.

Su **R402** (openvpn-client2) dentro a `root` è stato creato `client2.ovpn`:

```
client
dev tun
proto udp
remote 3.1.1.2 1194
resolv-retry infinite
ca ca.crt
cert client2.crt
key client2.key
remote-cert-tls server
cipher AES-256-GCM
```

Il file di configurazione è analogo a quello del client1.

A questo punto è possibile avviare la vpn con i seguenti comandi sui rispettivi nodi (nella directory in cui sono presenti i file di configurazione):

`openvpn server.ovpn`

`openvpn client1.ovpn`

`openvpn client2.ovpn`

Testing

Per verificare che le specifiche siano state correttamente rispettate, sono stati effettuati i seguenti test di raggiungibilità verso i diversi nodi della rete, utilizzando il ping.

Su **client-200**:
posso pingare le reti pubbliche

```
ping 2.1.1.2
ping 3.1.1.2
ping 4.1.1.2
```

se la vpn è attiva, è possibile pingare server e client della vpn (sono indicati gli indirizzi tipicamente scelti in modo automatico durante la configurazione della vpn, avendo definito l'indirizzo di rete):
`ping 192.168.150.1` -> verso GW300
`ping 192.168.150.10` -> verso R402
`ping 192.168.1.100` -> client-400 nella LAN dietro a R402

Inoltre è possibile raggiungere, sempre tramite la vpn, la rete del tenant B (non è possibile raggiungere la rete del tenant A)
`ping 10.0.20.10`
`ping 10.0.20.20`

Su GW300, client-400 e R402 sono stati condotti test analoghi.

Per verificare la corretta configurazione della DC Network sono stati realizzati i seguenti test.
Su **A1**:
posso pingare A2, ma non posso contattare le macchine del tenant B
`ping 10.0.10.20`

posso contattare le reti pubbliche esterne alla DC network (tranne 2.1.1.2 per le regole di firewall definite, ossia non accetta connessioni esterne alla LAN).

```
ping 3.1.1.2
ping 4.1.1.2
```

Test analoghi condotti per A2.

Su **B1**:
posso contattare B2, ma non le macchine del tenant A
`ping 10.0.20.20`

non posso contattare reti pubbliche esterne alla DC network direttamente, ma posso contattare l'access gateway
`ping 192.168.100.2`

Test analoghi condotti per B2.